Software Requirements Engineering & Specification

Unit 2: Requirements Gathering, Analysis, SRS & UML Modeling | Complete Exam Study Guide

STUDY GUIDE OVERVIEW & EXAM OBJECTIVE: This study guide provides an exhaustive, exam-ready reference for Software Requirements Engineering & Specification (Unit 2). It covers every topic and subtopic from the source materials in detail, including Elicitation Techniques, Functional vs Non-Functional Requirements (with 12 detailed quality attributes), the 7-Phase Requirements Engineering Process, SRS Structure & IEEE Template, SRS Characteristics, and all 7 UML & DFD Requirements Models with diagrams, rules, and examples. Each chapter features core concepts, step-by-step explanations, comparison tables, chapter summaries, exam points, solved questions, and MCQs with explanations.

CHAPTER 1: Requirement Gathering and Analysis

1.1 Definition and Core Purpose

Requirement Gathering and Analysis is the foundational initial phase of software development where developers and stakeholders work collaboratively to discover, understand, and document what the proposed software system must achieve. It involves collecting detailed information regarding user needs through structured techniques such as interviews, questionnaires, and observation.

Key goals of this phase include identifying both functional expectations (what the system should do) and non-functional constraints (performance, security, usability), analyzing raw requirements to detect conflicts, redundancies, or omissions, documenting findings in a formal **Software Requirements Specification (SRS)**, conducting a feasibility study to verify constraints, and obtaining formal stakeholder sign-off before proceeding to software design.

1.2 Stakeholders in Software Engineering

Stakeholder Definition: A stakeholder in software engineering is defined as any individual, group, or organization that is affected by, has a direct/indirect interest in, or can influence the development and eventual outcome of a software project.

Primary Stakeholder Categories & Roles:

• **End Users:** Individuals who will directly interact with the software daily (e.g., library members, bank customers, canteen students).

• **System Administrators / Operators:** Staff responsible for operational maintenance, server health monitoring, security, and user administration.

• **Software Engineers / Developers & Architects:** The technical team responsible for translating documented specifications into working code and architectural designs.

• **Project Sponsors & Senior Management:** Executives funding the project who focus on business ROI, project timelines, budget constraints, and strategic alignment.

1.3 Techniques for Collecting Requirements

Choosing the right elicitation technique is crucial for capturing accurate and complete requirements. The primary techniques outlined in the course material include:

Technique	Description & Key Characteristics	Advantages	Disadvantages / Trade-offs
1-on-1 Interviews	Direct face-to-face or virtual discussion with individual key stakeholders.	High depth of qualitative insights; builds rapport.	Time-consuming and expensive for large user bases.
Focus Groups & Workshops	Group discussions and facilitated brainstorming sessions with multiple stakeholders.	Faster consensus building; lower cost than 1-on-1 interviews.	Risk of groupthink or dominant personalities overpowering others.
Questionnaires & Surveys	Structured written/digital forms distributed to a wide sample of users.	Highly efficient for collecting quantitative data from large populations.	Limited depth; relies heavily on respondents providing honest, thorough info.
Observation (Job Shadowing)	Watching users perform their daily work processes and operational workflows.	Excellent for discovering unstated/hidden procedural requirements.	Can be intrusive; users may change behavior when observed (Hawthorne Effect).
Context Diagrams	High-level graphical visual modeling defining system boundaries and interfaces.	Clarifies external interactions and project scope clearly.	Does not detail internal system functions or data transformations.
Benchmarking	Comparing proposed practices or product traits against industry leaders or standards.	Generates innovation based on proven successful market solutions.	May not address unique organizational constraints or specific workflows.
Competitive Analysis	Evaluating competitor products to discover industry trends and missing features.	Identifies competitive differentiators and market standards.	Risk of copying competitor flaws or irrelevant feature bloat.

1.4 Feasibility Study

A **Feasibility Study** is an essential evaluation conducted during requirement gathering to assess whether the proposed software project is realistic, viable, and achievable within technical, financial, operational, and time constraints. It ensures that organization resources are not wasted on unachievable goals.

EXAM TIP: *Be prepared to contrast 1-on-1 Interviews with Focus Groups/Workshops regarding cost, speed, and qualitative depth. Always define 'Stakeholder' precisely using the official definition.*

Q1: What is a feasibility study and why is it conducted during requirement analysis?

Answer: A feasibility study analyzes whether a proposed system is technically, economically, and operationally achievable within budget, schedule, and resource limits before committing to full-scale development.

MCQ 1: Which requirement gathering technique is best suited for uncovering unstated daily workflow habits?

A) Questionnaires B) Observation C) Benchmarking D) Context Diagrams

Correct Answer: B) Observation. *Explanation: Watching users perform tasks reveals unstated routines and practical edge cases that users often forget to mention in written surveys or interviews.*

CHAPTER 2: Functional vs. Non-Functional Requirements

2.1 Functional Requirements (FRs)

Functional Requirements (FRs) specify the direct behaviors, operations, and services that a software system must perform. They outline *'WHAT'* the system should do in response to specific inputs, explicit operations, and error conditions.

Factors Influencing Functional Requirements:

• **Type of Software:** E.g., an e-commerce platform requires payment gateways, whereas an operating system requires hardware scheduling.

• **Target Users:** Technical administrators require detailed CLI/APIs, whereas general end-users require intuitive GUI elements.

• **Organizational Standards:** Document formatting and compliance policies enforced by the organization developing the software.

Concrete Example — University Library System:

• Users can search across all library databases or select specific database subsets.

• The system shall display requested documents using built-in document viewers.

• Every book/document order shall generate a unique **ORDER_ID** saved to the user's account storage.

2.2 Non-Functional Requirements (NFRs)

Non-Functional Requirements (NFRs) define the operational qualities, performance characteristics, and systemic constraints of the software. They specify *'HOW WELL'* the system performs its functions rather than specific user-facing actions.

Detailed Breakdown of 12 Quality Attributes (with Real-World Examples):

NFR Quality Attribute	Formal Definition & Core Focus	Real-World Industry Example	Measurable Requirement Statement
1. Availability	System accessibility whenever required by users.	Spotify 24x7 global streaming uptime.	The application shall maintain 99.95% uptime annually.
2. Reliability	Ability to perform functions correctly without failure.	Banking OTP generation working every time.	OTP verification shall execute without error 99.99% of requests.
3. Response Time	Latency or speed of system response to user input.	Spotify music search results loading.	Search queries shall return results within 2.0 seconds.
4. Scalability	Capacity to handle growing workload or user traffic.	IPL ticket booking managing massive bursts.	System shall support 100M simultaneous active users.
5. Security	Protection against unauthorized access & breaches.	Online banking multi-factor authentication.	User passwords shall be encrypted before database storage.
6. Maintainability	Ease of modifying, patching, or expanding software.	Swiggy adding new payment gateways smoothly.	New subscription tiers added without altering core services.
7. Recoverability	Ability to restore state & data after system crash.	Food app auto-restoring order details on crash.	System shall restore user playlists following unexpected crash.
8. Data Integrity	Ensuring data remains accurate, valid & consistent.	Google Pay bank balance consistency across failures.	Bank balance must never become inaccurate on tx failure.
9. Throughput	Volume of work/transactions processed per unit time.	Online payment gateway transaction processing.	System shall stream at least 50,000 songs/sec globally.
10. Usability	Ease with which new users learn and operate system.	Canteen food ordering without prior training.	New users shall create a playlist within 3 clicks.
11. Utilization	Efficiency of CPU, memory, and battery consumption.	Mobile app resource consumption limits.	App shall consume <100MB RAM & <5% battery/hr.
12. Manageability	Ease of administrative monitoring & governance.	Netflix server health monitoring dashboard.	Administrators shall monitor health via centralized dashboard.

2.3 Head-to-Head Comparison: FRs vs. NFRs

Comparison Dimension	Functional Requirements (FRs)	Non-Functional Requirements (NFRs)
Core Focus	System behaviors and operational features ('WHAT system does').	System quality attributes and constraints ('HOW WELL system performs').
Scope	Specific individual functions, inputs, and output logic.	Applies globally to the system as a whole.
Verification	Validated through functional & integration testing (Pass/Fail).	Validated through stress, load, performance, & security benchmarks.
Primary Example	User logs in with username/password; calculate order total.	System responds within 2 seconds; 99.9% uptime.

EXAM TIP: *Examiners love asking for 4-5 NFRs with concrete metrics. Never just write 'System must be fast'. Write 'System search response time shall be under 2 seconds.'*

MCQ 2: 'Passwords must be stored using SHA-256 encryption' is an example of which requirement category?

A) Functional Requirement B) Non-Functional Requirement (Security) C) Business Goal D) User Story

Correct Answer: B) Non-Functional Requirement (Security). *Explanation: Encryption specifies a security quality constraint and protection mechanism, which is a non-functional attribute.*

CHAPTER 3: Requirement Engineering Process

3.1 Concept and Role in SDLC

Requirements Engineering (RE) is a disciplined set of tasks and methods used to establish a deep, unambiguous understanding of what a software system should accomplish. It originates during the communication phase of software engineering and spans into the modeling phase, acting as an indispensable **bridge** between high-level stakeholder expectations and concrete architectural design & construction.

3.2 The 7 Phases of Requirements Engineering

The Requirements Engineering framework consists of 7 distinct, sequential yet iterative phases. Below is the complete step-by-step breakdown alongside the course's real-world **Travel Journey Analogy**:

Phase # & Name	Core Activity & Objective	Key Deliverables / Focus	Travel Journey Analogy
1. Inception	Establishes structural project baseline upon discovering a business need or market opportunity.	Problem domain definition; project scope baseline; team setup.	First initial idea of taking an adventurous motorcycling journey.

Phase # & Name	Core Activity & Objective	Key Deliverables / Focus	Travel Journey Analogy
2. Elicitation	Engages stakeholders to gather objectives, operational context, FRs, and NFRs.	Raw requirements list; stakeholder contact maps.	Researching the destination, talking to experienced riders, noting road challenges.
3. Elaboration	Refines raw requirements into structured models, classes, functions, and user interactions.	Class attributes, services, interactions, behavioral workflows.	Studying detailed route maps, analyzing road difficulties, weather patterns.
4. Negotiation	Resolves requirement conflicts among stakeholders through ranking and compromise.	Prioritized requirement matrix balancing cost, risk, & scope.	Discussing plans with riding companions, compromising on rest stops vs speed.
5. Specification	Documents compiled requirements into standardized formal artifacts (SRS document, prototypes).	SRS Document, usage scenarios, architectural prototypes.	Creating a printed detailed itinerary with fixed stops and travel times.
6. Validation	Reviews requirements work products for quality, completeness, consistency, and standards.	Audit sign-off, bug fixes in requirements, unambiguous analysis model.	Reviewing final itinerary with all riders to catch missing gear or roadblocks.
7. Management	Tracks, controls, and manages changes to requirements throughout the project lifecycle.	Traceability matrix, change control logs.	Adjusting riding routes dynamically during travel due to landslides or weather.

EXAM TIP: *Memorize all 7 RE phases in exact sequential order: Inception → Elicitation → Elaboration → Negotiation → Specification → Validation → Management. Questions often ask you to explain any 3 or 4 phases in detail.*

Q2: What is the main objective of the Negotiation phase in Requirements Engineering?

Answer: Negotiation resolves conflicting requirements among different stakeholders by ranking priorities, evaluating risks and development costs, and reaching mutually agreed compromises.

MCQ 3: Assessing requirement work products for consistency, clarity, and adherence to standards occurs in which phase?

A) Elaboration B) Specification C) Validation D) Management

Correct Answer: C) Validation. *Explanation: Validation is the quality assurance phase where requirements are audited for errors, ambiguity, and compliance prior to design.*

CHAPTER 4: Software Requirements Specification (SRS)

4.1 Concept and Diverse Audience

A **Software Requirements Specification (SRS)** is an authoritative document that comprehensively describes what a software system must accomplish before engineering development begins. It serves a diverse set of readers across the organization, each with distinct needs:

- **Senior Management & Sponsors:** To evaluate project feasibility, budget allocation, and strategic goals.
- **System Architects & Developers:** As the primary blueprint for building software architecture and writing source code.
- **Quality Assurance (QA) & Testers:** To construct test plans, test cases, and validation suites.
- **Maintenance Engineers:** To understand system design during post-deployment bug fixes and enhancements.
- **Clients & End Users:** To verify that their operational expectations are accurately understood.

4.2 IEEE Standard SRS Structure & Template Breakdown

The IEEE standard provides a globally recognized, highly structured framework for writing clear and consistent SRS documents. The standard template is structured into six core sections and supporting appendices:

SRS Section	Sub-sections & Components	Content Description & Scope
1. Introduction	1.1 Purpose, 1.2 Document Conventions, 1.3 Audience, 1.4 Scope, 1.5 References	Defines document purpose, reading suggestions, project scope boundaries, and reference citations.
2. Overall Description	2.1 Product Perspective, 2.2 Features, 2.3 User Classes, 2.4 Environment, 2.5 Constraints, 2.6 Docs, 2.7 Assumptions	High-level background: hardware/software context, user characteristics, operational environment, and design constraints.
3. System Features	3.1 System Feature 1, 3.2 System Feature 2 (Repeated for each feature)	Detailed breakdown of specific functional features, inputs, processing logic, and expected outputs.
4. External Interfaces	4.1 User Interfaces, 4.2 Hardware Interfaces, 4.3 Software Interfaces, 4.4 Communication Interfaces	Defines exact screen layouts, hardware protocols, API software hooks, and network communication protocols.
5. Other NFRs	5.1 Performance, 5.2 Safety, 5.3 Security, 5.4 Software Quality Attributes	Quantifiable non-functional constraints covering response times, encryption standards, and fault tolerance.
6. Other Requirements	Custom section for specialized domain requirements (e.g., legal or regulatory compliance).	Specialized database rules, internationalization, or legal compliance guidelines.
Appendices & Index	Appendix A: Glossary, Appendix B: Analysis Models, Appendix C: Issues List, Index	Definitions of acronyms, visual UML/DFD models, unresolved open issues, and index keywords.

4.3 Characteristics of a Good SRS Document

An SRS must adhere to seven essential quality characteristics to ensure it acts as a reliable software development contract:

Characteristic	Definition and Operational Meaning
1. Completeness	Contains all essential requirements, both functional and non-functional, leaving no blank areas.
2. Consistency	Free from internal contradictions or conflicting requirement statements.
3. Unambiguity	Every requirement has exactly ONE unambiguous interpretation, avoiding vague language.
4. Verifiability	Requirements are cost-effectively testable via demonstration, inspection, or automated tests.
5. Modifiability	Structure and style allow easy, consistent updates without breaking document integrity.
6. Traceability	Each requirement has a unique ID, traceable back to business needs and forward to code/tests.
7. Correctness	Accurately represents the true physical and operational needs of the client and stakeholders.

EXAM TIP: *A frequent 7-mark university exam question is 'Explain the characteristics of a good SRS document.' Be sure to list all 7 with brief definitions!*

MCQ 4: What characteristic ensures an SRS can be updated easily when requirements change?

A) Traceability B) Modifiability C) Verifiability D) Correctness

Correct Answer: B) Modifiability. *Explanation: Modifiability refers to an SRS document structure that allows changes to be made easily, completely, and consistently.*

CHAPTER 5: Requirement Modeling & UML Diagrams

5.1 Concept and Objectives of Requirement Modeling

Requirement Modeling is the practice of creating visual graphical representations of software requirements. It translates narrative textual needs into clear structural, behavioral, and flow diagrams.

Three Primary Objectives of Requirement Modeling:

1. To describe explicitly what the customer requires.
2. To establish a solid foundation for the creation of software design.
3. To define a clear set of requirements that can be validated once the software is built.

Five Categories of Requirement Models:

• **Scenario-based Models:** Viewpoints of system actors (e.g., Use Case diagrams, User Stories).

• **Class-oriented Models:** Object-oriented classes, attributes, operations, and collaborations (e.g., Class diagrams).

• **Behavioral & Patterns-based Models:** Dynamic behavior triggered by external events (e.g., State diagrams, Sequence diagrams).

• **Data Models:** Depict the information domain (e.g., E-R diagrams).

• **Flow-oriented Models:** Functional elements transforming data as it flows through system (e.g., Data Flow Diagrams - DFDs, Activity diagrams).

5.2 Detailed Study of 7 Requirements Diagrams

5.2.1 Use Case Diagram

A **Use Case Diagram** visually models user interactions with a system, highlighting actors and their goals. It provides a high-level view in layman's terms for stakeholders.

Component	UML Notation	Definition & Purpose
System Boundary	Rectangle enclosing use cases	Defines scope boundary; use cases sit inside, actors sit outside.
Actor	Stick Figure	External entity (human user, external hardware/system) interacting with system.
Use Case	Oval shape containing text	Represents a discrete user goal or function achieved via the system.
Association	Solid line connecting Actor & Use Case	Indicates that an actor participates in or triggers the use case.
Generalization	Solid line with hollow arrowhead	Represents inheritance between actors or use cases.
Include Relationship	Dashed arrow <<include>>	**Mandatory dependency.** Base use case ALWAYS executes included use case (e.g., CheckOrderStatus includes Login).
Extend Relationship	Dashed arrow <<extend>>	**Optional behavior.** Extending use case conditionally executes after base use case (e.g., PayByCard extends PayForItem).
Constraint	{condition} on dashed arrow	Shows specific conditional rule required for execution.

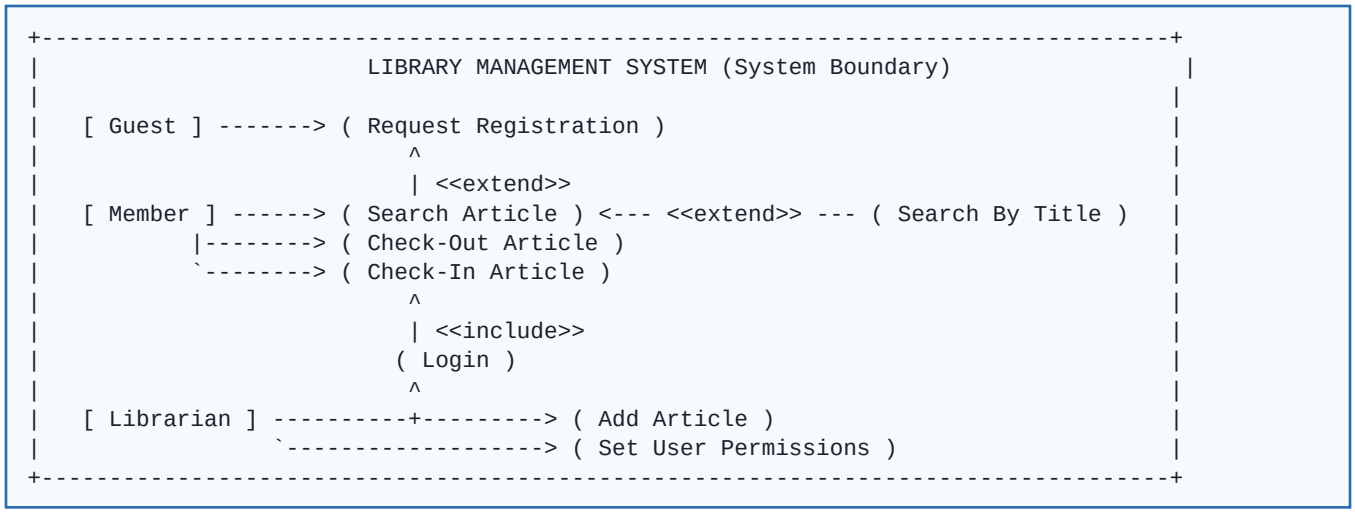
Use Case Diagram Guidelines & LMS Example:

Guidelines: Keep actors focused with a single coherent purpose; ensure each use case provides observable business value.

Library Management System (LMS) Actors & Use Cases:

• *Actors:* Librarian, Member, Guest.

• *Use Cases:* Login, Register User, Manage Permissions, Add/Delete Article, Check-In/Check-Out Article, Search Article (<<extend>> SearchByTitle, SearchByPublication).



5.2.2 Activity Diagram

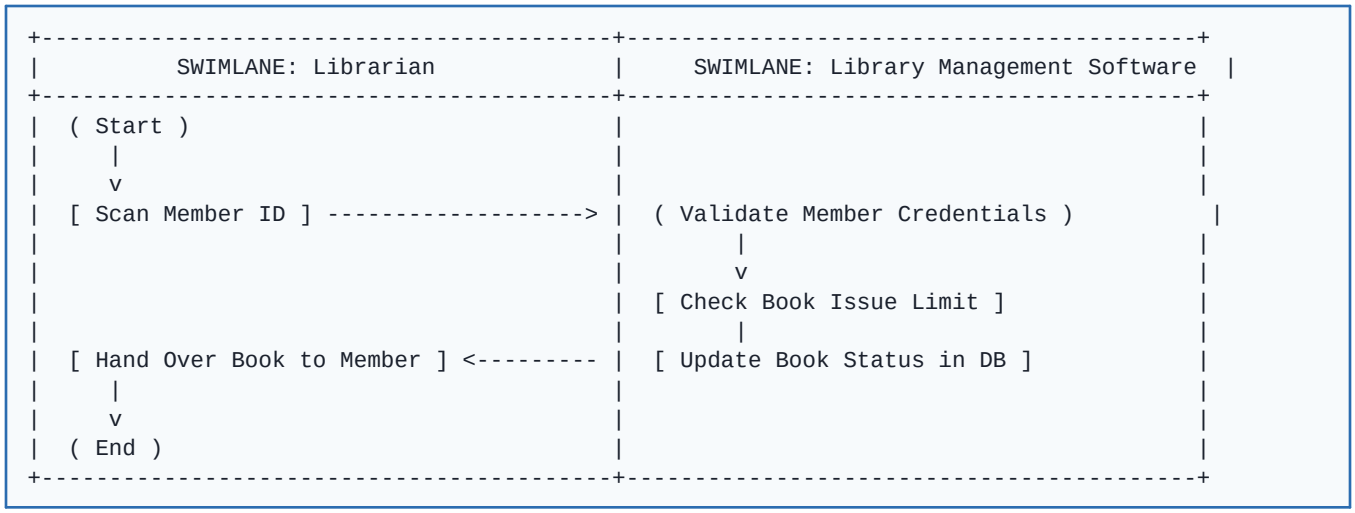
An **Activity Diagram** visually presents sequential and concurrent operational workflows (like a flowchart), focusing on *'HOW'* system operations execute.

Element	Symbol / Notation	Functionality
Activity State	Elongated Oval / Rounded Box	Represents an operation/function performed by system.
Control Flow	Unlabeled Arrow	Indicates completion of first activity and transition to next.
Initial State	Solid Black Circle	Starting entry point of the workflow.
Final State	Bull's Eye (Solid circle inside hollow circle)	Termination point of overall execution.
Branching	Diamond with outgoing condition arrows [true]/[false]	Decision point splitting flow based on guard conditions.
Fork & Join	Synchronization Bar (Heavy solid line)	Fork splits 1 incoming flow into concurrent parallel paths; Join merges them.

Book Issue Activity Workflow: Start → Check Availability → [Available] → Validate Member → Check Issue Limit → Add Transaction → Update Status → End.

5.2.3 Swimlane Diagram

A **Swimlane Diagram** is an activity diagram partitioned into vertical parallel columns ('swimlanes') representing departments, organizational roles, or hardware vs software entities responsible for executing specific activities.



5.2.4 Sequence Diagram

A **Sequence Diagram** models object interactions in explicit time sequence for a use case.

Element	UML Notation	Description
Object / Lifeline	Box with Object:Class + Vertical Dashed Line	Participant in interaction; dashed line represents existence over time.
Activation Box	Thin Vertical Rectangle on Lifeline	Indicates duration object is actively performing process or waiting.
Synchronous Message	Solid line with Solid Arrowhead	Sender blocks/waits for reply before continuing.
Asynchronous Message	Solid line with Lined Arrowhead	Sender continues execution without waiting for response.
Reply Message	Dashed line with Lined Arrowhead	Return value/response to prior call.
Delete Message	Solid arrow to an 'X'	Destroys object instance.

5.2.5 State Diagram

A **State Diagram** models the lifecycle of a single object as it transitions between states in response to external events. Transition syntax: Event(Attributes) [Condition] / Action.

State Machine Examples:
• **Book Object:** Available → [Book Issued] → Issued → [Return Request] → Returned → Available.
• **Bank ATM:** Off → [Turn On] → Self Test → [Pass] → Idle → [Insert Card] → Serving Customer (Substates: Auth, Select Tx, Tx) → [Eject] → Idle.

5.2.6 Class Diagram

A **Class Diagram** provides an object-oriented structural blueprint, showing classes, attributes, operations, and relationships.

3 Compartments & Access Modifiers:
1. *Top:* Class Name (Bold, centered; Italic for Abstract Class).
2. *Middle:* Attributes (accessModifier name:dataType = defaultValue).
3. *Bottom:* Operations (accessModifier methodName(argList):returnType).
Access Modifiers: Public (+), Private (-), Protected (#), Package (~), Static (underlined).

Relationship	UML Symbol / Line	Semantic Meaning & Example
Generalization	Solid line + Hollow Arrowhead	'Is-A' inheritance (e.g., Car IS-A Vehicle; SavingAccount IS-AN Account).
Aggregation	Solid line + Hollow Diamond	'Has-A' weak ownership. Child exists independently of parent (e.g., Car HAS-A Wheel).
Composition	Solid line + Solid Diamond	'Part-Of' strong ownership. Child destroyed if parent destroyed (e.g., Person & Brain/Heart).
Multiplicity	Labels like 0..1, 1, *, 1..*	Defines bounds on instances related (e.g., Bank 1 --- 1..* Branch).

5.2.7 Data Flow Diagram (DFD)

A **Data Flow Diagram (DFD)** graphically tracks how data moves through an information system.

DFD Element	Gane & Sarson Notation	Yourdon & Coad Notation	Function
External Entity	Rectangle (Double border)	Circle or Square	Source/destination of external data.
Process	Rounded Rectangle (Horizontal line)	Circle (Bubble)	Transforms incoming data flows into outputs.
Data Store	Open-ended Rectangle	Parallel Lines	Holds data at rest for later access.
Data Flow	Labeled Arrow	Labeled Arrow	Movement of data between elements.

3 Mandatory Rules for DFD Design:

• **Rule 1:** No process can have ONLY inputs or ONLY outputs ('black holes' or 'miracle processes' are invalid).

• **Rule 2:** Data cannot flow directly between an External Entity and a Data Store; it MUST pass through a Process.

• **Rule 3:** Data cannot flow directly between two External Entities without passing through a Process.

DFD Hierarchy Levels:
• **Level-0 (Context Diagram):** Represents entire system as a single high-level process interacting with external entities.
• **Level-1 DFD:** Decomposes the single Level-0 process into main functional sub-processes (1.0 Issue Book, 2.0 Return Book, 3.0 Search Book).

EXAM TIP: *Practice drawing Include vs Extend relationships, Aggregation vs Composition symbols, and Level-0 vs Level-1 DFDs. These are guaranteed exam drawing questions!*

MCQ 5: If deleting a 'House' object automatically destroys all associated 'Room' objects, which UML relationship is present?

A) Generalization B) Aggregation C) Composition D) Association

Correct Answer: C) Composition. *Explanation: Composition represents strong 'part-of' ownership where lifecycle of the child object is strictly dependent on the parent object.*

CHAPTER 6: Final Revision & Exam Cheat Sheet

6.1 Core Definitions Cheat Sheet

Key Term	One-Sentence Exam Definition
Requirements Analysis	The initial process of uncovering, analyzing, and documenting stakeholder expectations for a system.
Stakeholder	Any individual, group, or organization affected by, interested in, or capable of influencing a software project.
Functional Requirement	A statement specifying a concrete behavior, feature, or operation that the system must perform.
Non-Functional Requirement	A constraint specifying quality attributes (e.g., speed, security, uptime) of the software.
Requirements Engineering	Disciplined 7-phase process bridging stakeholder needs with architectural software design.
SRS Document	Formal authoritative contract specifying user & system requirements prior to development.
Use Case Diagram	High-level UML model showing actor interactions with system functions inside a system boundary.
Sequence Diagram	Dynamic UML behavioral diagram illustrating message exchange sequences between objects over time.
State Diagram	Graph modeling object lifecycle state changes triggered by external stimuli/events.
Composition	Strong UML aggregation where child objects cannot exist independently of parent objects.
Context Diagram (Level-0 DFD)	Top-level DFD depicting entire software system as one single process linked to external entities.

6.2 Master Comparison Matrix

Concept A	Concept B	Key Distinguishing Difference
Functional Requirement (FR)	Non-Functional Requirement (NFR)	FR = 'WHAT system does'; NFR = 'HOW WELL system performs'.
<> Use Case	<> Use Case	<> is mandatory execution; <> is optional conditional execution.
Aggregation	Composition	Aggregation = weak 'has-a' (child survives parent); Composition = strong 'part-of' (child dies with parent).
Synchronous Message	Asynchronous Message	Sync = sender blocks waiting for reply; Async = sender continues immediately.
Level-0 DFD (Context)	Level-1 DFD	Level-0 shows 1 system bubble; Level-1 breaks down into functional sub-processes.
Activity Diagram	Sequence Diagram	Activity = operational workflow sequence; Sequence = message exchange between objects over time.

6.3 Golden Rules for Exam Success

• **Rule 1:** Always write measurable metrics for NFR questions (e.g., '99.9% uptime', '<2 seconds response time').

• **Rule 2:** Draw complete UML diagrams with boundaries, actor names, arrows, and stereotype labels (<<include>>).

• **Rule 3:** When designing DFDs, ensure every process has at least one input and one output.

• **Rule 4:** Remember the 7 RE phases in sequence: Inception, Elicitation, Elaboration, Negotiation, Specification, Validation, Management.

• **Rule 5:** Remember the 7 SRS characteristics: Completeness, Consistency, Unambiguity, Verifiability, Modifiability, Traceability, Correctness.

> **FINAL EXAM READINESS CHECKLIST: CONGRATULATIONS!** You have completed the comprehensive study guide for Software Requirements Engineering & Specification (Unit 2). Review these core concepts, tables, and exam points prior to your examination for complete readiness!